

Sri Lanka Institute of Information Technology



IT5072 - Natural Language Processing [2026/JUL]

MSc in AI (Batch - 02)

Coursework – 01

Statistical POS-Tagged Parser

Using the Penn Treebank English Corpus

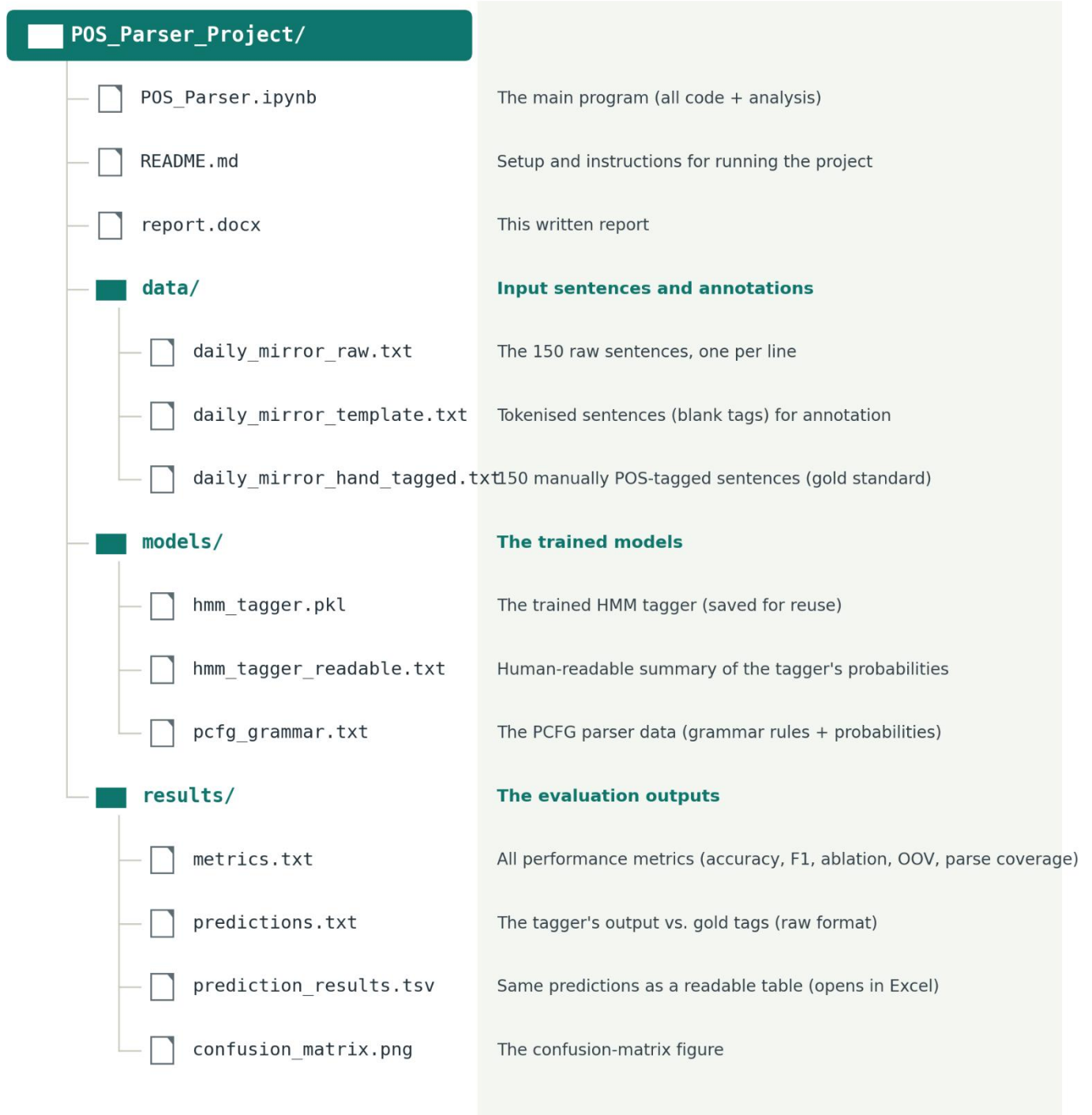
Project Report

Name: K. A. D. B. SANDARUWAN

Index : MS26916262

0. Project Structure

The project is organized into four folders (data, models, results, and report) together with the main program notebook and a README. The complete structure is shown below.



Folder Overview

Directory	Contents
data/	Contains the test data. daily_mirror_raw.txt stores the original sentences collected from the newspaper, daily_mirror_template.txt contains the tokenized text prepared for tagging, and daily_mirror_hand_tagged.txt contains the manually annotated gold standard used to evaluate the tagger.
models/	Contains the trained models. hmm_tagger.pkl is the trained HMM tagger saved for reuse, hmm_tagger_readable.txt is a human-readable summary of its transition and emission probabilities, and pcfg_grammar.txt is the PCFG parser data, the grammar rules with their probabilities.
results/	Contains the evaluation outputs. metrics.txt holds all performance metrics (accuracy, F1, ablation, and OOV analysis), predictions.txt and prediction_results.tsv hold the tagger's predictions against the gold tags (in raw and readable-table form), and confusion_matrix.png is the confusion-matrix figure.

The development of this project was tracked on GitHub:

<https://github.com/BimsaraS99/statistical-pos-tagger-pcfg-parser>

1. Introduction

1.1 Background

Part-of-Speech (POS) tagging assigns a category to each word according to its context. Tags include noun (NN), verb (VB), adjective (JJ), and adverb (RB). In “The cat sleeps”, “cat” is a noun and “sleeps” is a verb. POS tagging is fundamental in natural language processing (NLP) because it supports tasks such as parsing and information extraction. Early POS taggers used hand-written linguistic rules, but natural language contains ambiguity and irregularity, limiting their ability to generalize. A major challenge is lexical ambiguity, when the same word can have different tags depending on context. For instance, “book” is a noun in “read a book” but a verb in “book a flight”. Statistical methods address this challenge by learning tag probabilities from large annotated corpora, making tagging more robust to ambiguity.

Syntactic parsing determines the structure of a sentence by grouping words into phrases, such as noun and verb phrases. This is modelled using a Probabilistic Context-Free Grammar (PCFG), which assigns probabilities to grammar rules estimated from annotated parse trees in a treebank. This report develops statistical parser combining POS tagging and PCFG parsing, trained on the Penn Treebank and evaluated on news text.

1.2 Objectives

This main objective of this assignment is to design and implement a statistical part-of-speech tagger and parser. Both components are developed and evaluated using the Penn Treebank corpus. The objectives are:

- To build a statistical POS tagger using transition and emission probabilities and Viterbi decoding.
- To induce a probabilistic context-free grammar (PCFG) parser by extracting grammar rules from the Penn Treebank trees.
- To collect 150 sentences from the Daily Mirror online newspaper and manually tag them as a gold standard.
- To evaluate tagging performance using standard evaluation metrics and analyze the effect of domain differences between the training and test data.

1.3 Scope

This report focuses on POS-tagging evaluation, measuring how accurately the tagger labels the individual words across the 150 Daily Mirror sentences using standard metrics such as accuracy, precision and recall. The PCFG parser is demonstrated by parsing a sample of sentences, but a full parse-tree evaluation is not performed, since gold-standard parse trees were not manually annotated for the test data, only POS tags. The study is therefore limited to real-world English news text and does not cover other domains.

2. Dataset

2.1 Penn Treebank Corpus (NLTK Labeled)

The training data for this project is the Penn Treebank corpus, a widely used annotated corpus in natural language processing. The full Penn Treebank contains approximately 1 million words of English text, in which every word is labelled with a POS tag and every sentence is annotated with its syntactic structure. As instructed, this project uses the sample version of the Penn Treebank provided through the NLTK library, which is a subset of the full corpus containing around 3900 sentences. This sample provides both the annotated word/tag pair required to train the tagger and the annotated parse trees required to induce the PCFG parser.

2.2 Corpus Statistics

This below table summarizes the key statistics of the training corpus.

Metric	Value
Number of sentences	3,914
Number of tokens	100,676
Distinct words	11,387
Distinct POS tags	46
Average sentence length	25.7 words

3. Methodology

3.1 Statistical POS Tagging (HMM)

The POS tagger that we have used here is based on a Hidden Markov Model (HMM), a probabilistic sequence model widely used for tagging. In an HMM, the words of a sentence are treated as observed variables, while their POS tags are treated as hidden (unobserved) variables that must be inferred.

The model assumes that the tag of each word depends only on the previous tag (Markov assumption) and that each word is generated from its own tag.

Tagging a sentence therefore becomes the problem of finding the most probable sequence of hidden tags given the observed sequence of words.

Formally, for a sentence of words $w_1, w_2, \dots, w_n$, the goal is to find the tag sequence $t_1, t_2, \dots, t_n$, that maximizes the joint probability, as shown in the equation that I have given below.

$$\hat{t}_1^n = \arg \max_{t_1^n} \prod_{i=1}^n P(t_i | t_{i-1}) P(w_i | t_i)$$

Here, $P(t_i | t_{i-1})$ is the transition probability and $P(w_i | t_i)$ is the emission probability, these two methods are discussed below.

3.2 Transition and Emission Probabilities

The two probability distribution of the HMM are estimated from the tagged training corpus, the Penn Treebank corpus using Maximum Likelihood Estimation (MLE), which computes each probability directly from frequency counts in the corpus.

The transition probability is the probability of a tag given previous tag. It captures syntactic patterns. For example, a noun frequently follows a determiner. This is estimated as the count of a tag pair divided by the count of the preceding tag, as I have given in the below equation. Here $C(t_{i-1}, t_i)$ denotes the number of times tag t_i follows tag t_{i-1} .

$$P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

The emission probability is the probability of a word given its tag. It captures the association between a word and a POS tag. For example, the word ‘dog’ is far more likely under tag NN

than under VB. It is estimated as the count of a word-tag pair divided by the total count of that tag, as we have given in the below equation. Here $C(t_i, w_i)$ the number of times word w_i is tagged t_i .

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

Both distributions are obtained purely by counting occurrences in the annotated corpus (Penn Treebank), that is defining characteristic of the statistical approach.

3.3 Viterbi Decoding

Once the transition and emission probabilities are estimated, the tagger must find the most probable tag sequence for a new sentence. Evaluating every possible tag sequence is computationally infeasible, since a sentence of n words with T possible tag has T^n possible taggings. The Viterbi algorithm solves this efficiently using dynamic programming.

Viterbi algorithm process the sentence from the left to right, and at each word it stores, for every possible tag, the probability of the most probable path that ends in that tag. Let $v_t(j)$ denote the highest probability of any tag sequence ending in tag j at position t . This value is computed recursively from the best paths at the previous position, as shown in the equation that I have given below.

$$v_t(j) = \max_i [v_{t-1}(i) \cdot P(t_j|t_i)] \cdot P(w_t|t_j)$$

By keeping a back-pointer to the previous tag that produced each maximum, the algorithm can trace back the single most probable tag sequence once the end of the sentence is reached. Since it reuses previously computed values instead of recomputing them, Viterbi reduces the complexity from $O(T^n)$ to a value linear in sentence length. In this implementation, probabilities are computed in log space (summing log-probabilities) to avoid underflow when multiplying many small values.

3.4 PCFG Parsing

While the HMM tagger labels individual words, the parser recovers a sentence's syntactic structure using a Probabilistic Context-Free Grammar (PCFG). A PCFG extends a context-free grammar, consisting of non-terminals (phrase categories such as S, NP, VP), terminals (words), and production rules $A \rightarrow \beta$. It does so by assigning a probability to each rule.

For every non-terminal, the probabilities of its alternative rules sum to 1. Each rule probability is estimated from the treebank by MLE. The equation for the rule probability we have given below. Here $C(A \rightarrow \cdot)$ is the total count of rule with A on the left-hand side.

$$P(A \rightarrow \beta) = \frac{C(A \rightarrow \beta)}{C(A \rightarrow \cdot)}$$

The probability of a complete parse tree is the product of the probabilities of all rules used to derive it. The tree probability formula is given below.

$$P(\text{tree}) = \prod_{r \in \text{tree}} P(r)$$

When a sentence has multiple valid parses, the parser selects the tree with the highest probability, thereby resolving structural ambiguity.

3.5 Chomsky Normal Form and CKY

The parser uses the CKY (Cocke–Younger–Kasami) algorithm, which requires the grammar to be in Chomsky Normal Form (CNF). In CNF, every rule must have exactly two non-terminals on its right-hand side or produce a single terminal.

Since treebank rules are often wider (For example: $NP \rightarrow DT \quad JJ \quad NN$), they are converted to CNF by binarization, which splits each wide rule into binary rules using auxiliary symbols. CKY parses bottom-up: it labels individual words, then repeatedly combines pairs of adjacent constituents into larger phrases until the whole sentence is derived as S, keeping the highest-probability constituent for each span. Its complexity is approximately $O(n^3)$ in sentence length, which is efficient for the short sentences used here.

3.6 Smoothing and Unknown-Word Handling

A central challenge in applying a tagger trained on the Penn Treebank to Daily Mirror news Online article’s text is the high proportion of words unseen during training. In this section we describe the smoothing and unknown-word handling strategy that we used to achieve high accuracy on unknown words. Although based on established techniques, it was designed specifically for this out-of-domain setting and proved to be the most influential component of the system, as demonstrated in the ablation study (Section 6.3).

Smoothing:

Maximum likelihood estimation assigns zero probability to any word–tag or tag–tag pair not observed in training. Since these probabilities are multiplied along a sentence, a single zero would reduce the probability of the entire sequence to zero. To prevent this, Laplace (add-one) smoothing is applied, adding one to every count before normalization, as shown in Equation below. This guarantees that every event receives a small non-zero probability.

$$P(\mathbf{w}_i | \mathbf{t}_i) = \frac{C(\mathbf{t}_i, \mathbf{w}_i) + 1}{C(\mathbf{t}_i) + V}$$

Unknown-word handling:

For words that are not available in the training vocabulary, emission probabilities cannot be estimated directly. Rather than assigning a uniform fallback, we have used a different method. This project adopts a data driven suffix-based strategy: suffix-to-tag distribution is learned from low-frequency words in the training corpus (those occurring fewer than a set number of times), on the assumption that rare words behave similarly to unseen ones. An unknown word is assigned likely tag based on its suffix. For an example, words ending “-ing” are usually tagged as VBG. This is combined with two other cues such as capitalized words are more likely to be NNP (proper nouns), while words containing digits are more likely to be CD (cardinal numbers). The mechanism can be switched on or off to measure its effect in the ablation study (Section 6.3).

4. Implementation

4.1 Tools and Environment

Component	Library/Tool
Programming Languages	Python 3
Corpus access, tokenization, POS tagging, parsing.	NLTK
Evaluation metrics (precision, recall, F1, confusion matrix)	Scikit-learn
Numerical operations	NumPy
Figure and plots	matplotlib
Model persistence	pickle

4.2 Training the Tagger

The tagger was trained by counting tag-to-tag transitions and word-to-tag emissions across the tagged training corpus, then normalizing these counts into probabilities. Add-one smoothing and the learned suffix-based unknown-word mechanism (section 3.6) were applied at the probability-lookup stage. The most probable tag sequence for each sentence was decoded using a log-space implementation of the Viterbi algorithm to ensure numerical stability.

4.3 Building the PCFG Parser

The PCFG was induced from the parsed trees of the training corpus. Each tree's production rules were extracted, function tags (e.g., NP-SBJ) were removed to reduce grammar sparsity, and the trees were converted to Chomsky Normal Form (CNF) so that all rules were binary. The rules were then counted and normalized into probabilities using NLTK's grammar-induction utilities, and parsing was performed using a Viterbi (CKY-based) parser.

4.4 Tagging the 150 Sentences

The trained tagger was applied to the 150 tokenized Daily Mirror sentences to generate predicted tags. These predictions were aligned token-by-token with the manually annotated gold tags, forming the basis for the evaluation presented in Section 6.

5. Data Collection and Annotation

5.1 Source: Daily Mirror Online

The test data consist of 150 sentences manually collected from the Daily Mirror online newspaper (dailymirror.lk). Sentences were selected by hand from published news articles randomly to ensure the evaluation covered a range of language styles and vocabulary, and were drawn from four sections: business, politics, sport and local news. Each sentence was chosen to be complete and grammatically well-formed, while headlines, fragments, and sentences dominated by numbers or symbols were deliberately avoided, as these do not follow standard sentence structure.

5.2 Daily Mirror Dataset Statistics

Metric	Value
Number of sentences	150
Number of tokens	2451
Distinct words	1073
Distinct POS tags	36
Average sentence length	16.3 words
Sentences per section	Business 40, politics 40, sport 40, local 30

5.3 Tokenization

Before tagging, each sentence was tokenized using the Penn Treebank tokenization conventions, via NLTK's *TreebankWordTokenizer*, so that the test tokens matched the format of the training data. Punctuation is separated into individual tokens, and contractions are split. For example, "don't" becomes "do" and "n't", while possessive "'s" is separated from the noun. This consistent tokenization allows a fair comparison between the model's predictions and the training vocabulary.

5.4 Manual Tagging Criteria

Each token was manually assigned a POS tag from the 36-tag Penn Treebank tagset (the complete set of POS tags used by a tagger.), producing a gold-standard reference for evaluation. Particular care was taken with ambiguous cases, such as distinguishing VBD (past tense) from VBN (past participle), common nouns (NN) from proper nouns (NNP), and adjectives (JJ) from participles. This hand-annotated set serves as the ground truth against which the tagger's predictions are compared.

6. Results and Evaluation

6.1 Evaluation Methodology

The tagger was evaluated by comparing its predicted tags against the manually annotated gold standard on a token-by-token basis across all 2,451 tokens. Several standard performance metrics were used to evaluate the tagger. Per-token accuracy measures the percentage of individual words that were assigned the correct POS tag, while sentence-level accuracy measures the percentage of sentences in which every word was tagged correctly. A most-frequent-tag baseline was included as a reference point, and an ablation study was conducted to examine the contribution of the unknown-word handling mechanism. Performance was also analyzed using per-section accuracy, per-tag precision, recall, and F1-score, together with a confusion matrix and out-of-vocabulary (OOV) analysis.

6.2 Overall Accuracy and Baseline

Metric	Value
Per-token (word-level) accuracy	83.52% (2047 / 2451)
Overall Precision (weighted)	0.852
Overall Recall (weighted)	0.835
Overall F1-score (weighted)	0.836
Sentence-level accuracy	9.33% (14 / 150 fully correct)
Baseline accuracy (most-frequent tag)	79.89%
Total tokens evaluated	2,451
OOV rate	13.71% (336 / 2451)

The tagger achieved a per-token accuracy of 83.52% and a weighted F1-score of 0.836, both above the 79.89% baseline. Sentence-level accuracy was only 9.33%, since a single mistagged word causes an entire sentence to be counted as incorrect.

6.3 Model Comparison (Ablation Study)

Configuration	Accuracy
1. Baseline (most-frequent tag)	79.89%
2. HMM + Viterbi (clues off)	75.68%
3. Full model (clues on)	83.52%
Context gain (1 $\rightarrow$ 2)	-4.20%
Unknown-word gain (2 $\rightarrow$ 3)	+7.83%
Total gain (1 $\rightarrow$ 3)	+3.63%

In this study I wanted to know the contribution of each component of the pipeline for the accuracy. Therefore, the ablation study isolates the contribution of each component by evaluating three configurations. The baseline assigns each word its most frequent training tag, ignoring context, and achieves 79.89%.

The “clues off” configuration is the full HMM with Viterbi decoding but with the *unknown-word mechanism disabled*, so out-of-vocabulary words receive only a uniform fallback.

The “clues on” configuration is the complete model, in which unseen words are handled using the learned suffix distribution and orthographic cues described in Section 3.6.

A notable result is that the “clues off” configuration (75.68%) performed below the baseline, a drop of 4.20 percentage points. Although Viterbi decoding adds contextual information, on this out-of-domain data the uninformative guesses assigned to the many unseen words propagate through the tag sequence and degrade neighboring predictions, outweighing the benefit of context. Enabling the unknown-word mechanism reversed this entirely, raising accuracy to 83.52% with a gain of 7.83 percentage points. And also, it is the largest single improvement in the study. This shows that, for out-of-domain news text with a high OOV rate, effective unknown-word handling had a greater impact on the tagger’s performance than contextual decoding alone.

6.4 Per-Section Accuracy

Section	Accuracy	Correct / Total
Business	82.91%	553 / 667
Politics	84.13%	578 / 687
Sport	84.78%	557 / 657
Local	81.59%	359 / 440

Accuracy was highest on sport (84.78%) and lowest on local news (81.59%). The lower local-news performance reflects its higher concentration of out-of-vocabulary like Sri Lankan names absent from the training corpus.

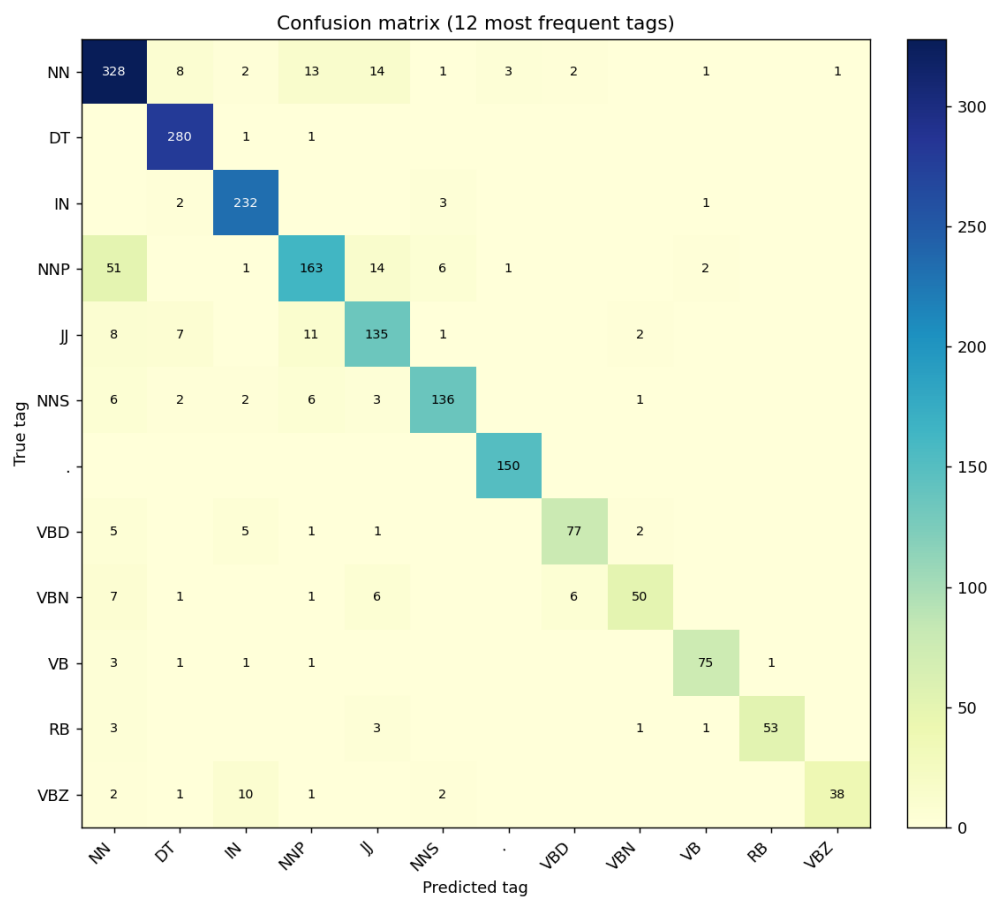
6.5 Per-Tag Precision, Recall and F1

Below table presents the per-tag metrics for the most frequent tags, the complete report is provided in *results/metrics.txt*.

POS Tag	Precision	Recall	F1	Support
CC	1.000	1.000	1.000	54
CD	0.905	0.927	0.916	41
DT	0.900	0.993	0.944	282
IN	0.866	0.892	0.879	260
JJ	0.714	0.789	0.750	171
MD	0.941	0.914	0.928	35
NN	0.783	0.861	0.820	381
NNP	0.787	0.665	0.721	245
NNS	0.901	0.834	0.866	163
POS	0.905	1.000	0.950	19
PRP	0.903	0.966	0.933	29

PRP\$	1.000	1.000	1.000	15
RB	0.964	0.791	0.869	67
TO	0.716	1.000	0.835	53
VB	0.915	0.882	0.898	85
VBD	0.906	0.770	0.832	100
VBG	0.870	0.455	0.597	44
VCN	0.862	0.588	0.699	85
VBP	0.842	0.696	0.762	23
VBZ	0.974	0.655	0.784	58
.	0.968	1.000	0.984	150
,	0.808	1.000	0.894	42
Macro avg	0.623	0.529	0.550	2451
Weighted avg	0.852	0.835	0.836	2451

6.6 Confusion Matrix



The most frequent error was the misclassification of 51 proper nouns (NNP) as common noun (NN). Verb form (VBD, VBN, VBG) were also confused with one another and with adjectives and nouns.

6.7 Out-of-Vocabulary Analysis

Category	Count	Percentage
Total tokens evaluated	2,451	100.00%
In-vocabulary (seen) words	2,115	86.29%
Out-of-vocabulary (OOV) words	336	13.71%
Accuracy on seen words	1785 / 2115	84.40%
Accuracy on OOV words	262 / 336	77.98%

Although 13.71% token was out-of-vocabulary, accuracy on these words (77.98%) was only 6.42 points below seen-word accuracy (84.40%), showing that the suffix and orthographic clues recovered most unseen words. Remaining errors included named entities ('Google', 'Programme', tagged NN) and dialectal words ('whilst', 'towards').

6.8 Parsing Results

The PCFG parser was applied to all 150 test sentences, of which only 2 were parsed successful (1.3% coverage).

This low coverage is expected. Unlike tagging, a PCFG parse requires every word and the whole sentence structure to be covered by the grammar, so a single out-of-vocabulary word or unseen structure prevents the entire sentence from being parsed. As the news sentences contain many such words, almost all failed. A formal PARSEVAL evaluation was not performed, as gold-standard parse trees were not annotated for the test data.

7. Error Analysis and Discussion

7.1 Domain Mismatch

A fundamental source of error was the mismatch between the training and test domains. The tagger was trained on the Penn Treebank, consisting of 1989 Wall Street Journal financial news, whereas the test data comprised modern Sri Lankan news. These domains differ in topic/subject matter, vocabulary and even sentence length (16.3 words on average versus 25.7 in training). As a result, many test words were either unseen or used in unfamiliar contexts. This is also reflected in the per-section results, with the local news section, which contains the most local content, achieving the lowest accuracy of 81.59%.

7.2 Unknown Words and Named Entities

Proper Noun Errors The tagger's weakest area was proper noun recognition. NNP achieved a recall of only 0.665, with 51 proper nouns misclassified as common nouns (NN). These errors mainly involved unseen names such as “Colombo”, “Google”, and “Programme”. The unknown-word mechanism achieved 77.98% OOV accuracy using suffix and capitalization cues, but these were not always sufficient, particularly at sentence starts or within multi-word names. Richer named-entity features could further improve this.

7.3 Tokenization Effects

A smaller number of errors were related to tokenization. Hyphenated compounds such as “rip-and-replace”, which function as adjectives (JJ), were sometimes tagged as common nouns (NN) because they were rare in the training data. Possessive constructions were generally handled correctly, as the tokenizer separated the ‘s clitic according to the Penn Treebank convention.

7.4 Common Tag Confusions

Beyond proper nouns, the most frequent confusions involved verb inflections. The tagger often confused VBD, VBN and VBG with one another and with adjectives or nouns. For example, VBG recall was only 0.455. This reflects the ambiguity of ‘-ing’ and ‘-ed’ forms, which can function as verbs, adjectives, or nouns depending on the context. For example, an ‘-ing’ word may be a present participle (VBG), an adjective (JJ), or a gerund noun (NN).

7.5 Closed-Class vs Open-Class Performance

A clear difference was seen between closed- and open-class categories. Closed-class tags, such as CC, DT, PRP and MD, achieved near-perfect F1 scores because they contain a small set of common words that are well represented in the training data. In contrast, open-class categories, such as proper nouns, adjectives and verbs, performed worse because they can continuously gain new words, leading to more unseen vocabulary. This suggests that most errors were related to vocabulary coverage rather than grammatical knowledge.

8. Limitations

This study has several limitations. The NLTK Penn Treebank sample (~3,900 sentences) is small, producing sparse statistics and a high out-of-vocabulary rate on new text. There is also a clear domain mismatch between the training data (1989 US financial news) and the test data (modern Sri Lankan news). Furthermore, evaluation was limited to POS-tagging metrics, as gold-standard parse trees were not available for a full PARSEVAL evaluation of the parser. Finally, the bigram HMM is a relatively simple model; CRF or neural taggers would likely perform better on unseen vocabulary

9. Conclusion

This project implemented a statistical HMM POS tagger with Viterbi decoding and a PCFG parser, trained on the Penn Treebank and evaluated on 150 manually annotated Daily Mirror sentences. The tagger achieved 83.52% per-token accuracy (weighted F1 0.836), exceeding the baseline by 3.63 percentage points. The analysis identified domain mismatch as the main challenge, and the ablation study showed that robust unknown-word handling was the most important factor in the system's performance